TD/TP9 : Correction des exercices de TD

CPES « Sciences des données, arts et cultures » : Introduction à l'algorithmique

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

12 mars 2024

Question 1 On considère un digraphe D représenté par une *liste d'adjacence*, c'est à dire, où chaque sommet est associé à la liste de ses voisins sortants. Combien de temps (en notation asymptotique) faut-il pour :

— Calculer les degrés sortants de tous les sommets ?

— Calculer les degrés entrants de tous les sommets ?

Justifier votre résultat en détaillant les différentes étapes de votre raisonnement.

Correction : Il y a donc deux cas de figure à traiter. Les voici :

— *Degrés sortants.* Comme aucune hypothèse n'est faite sur l'implémentation des listes, nous n'avons pas la garantie que la taille de la liste est mise à jour et donc accessible en $O(1)$. De ce fait, pour connaître le degré sortant de chacun des sommets de $D = (V, E)$, il faut parcourir la liste des voisins de chacun des sommets de V. Cela a un coût total de $O(|E|)$, avec $|E|$ le nombre d'arêtes de D. Il faut également faire attention à prendre en compte de potentiels sommets qui auraient une liste d'adjacence vide, donc ne contenant aucun voisin. Vérifier qu'une liste est vide se fait aisément en $O(1)$. Ainsi, pour couvrir tous les cas de figure possibles, on peut écrire la complexité temporelle en $O(|E| + |V|)$, où $|V|$ est le nombre de sommets de D.

— *Degrés entrants.* Dans ce cas, il n'est pas possible d'accéder directement à la liste des « voisins entrants »directement pour chaque sommet. Il va falloir parcourir la liste de voisins de chaque sommet de D, et compter le nombre de fois qu'apparaît chaque sommet. Le maintient de $|V|$ compteurs se fait en $O(1)$ à chaque mise à jour, et on aura en tout $|E|$ mises à jour (une par arête). Cela fait donc une complexité temporelle également en $O(|E| + |V|)$, qui couvre aussi le cas où l'on aurait des listes de voisins vides pour tout ou partie des sommets V.

Question 2 Montrer qu'un sommet u ayant un degré entrant et un degré sortant strictements positifs, peut se retrouver seul dans une composante connexe de la forêt DFS. Pour rappel, la forêt DFS est définie par $D = (V, E_\pi)$, où $E_\pi = \{(u.\pi, u) : u \in V\}$.

Correction : La clé ici réside dans l'ordre de visite des sommets par DFS. Il suffit de donner un exemple pour lequel on vérifie la situation exposée. Prenons un digraphe $D = (V, E)$,

avec $V = \{u, x, y\}$ et $E = \{(x, u), (u, y)\}$. Concrètement, cela correspond au digraphe suivant : $x \to u \to y$. On a bien ici un sommet u atant un degré entrant et un degré sortant strictements positifs (valant 1 dans les deux cas). On peut dès lors procéder dans cet ordre :

— Appliquer DFS(y). Comme aucun autre sommet n'est accessible depuis y, le parcours s'arrête immédiatement. La forêt DFS (initialement vide) est alors maintenant composée d'un seul noeud y, formant de fait une composante connexe seul.

— Appliquer DFS(u). Ici le seul sommet accessible y est déjà visité, donc le parcours s'arrête aussitôt. On ajoute alors u à la forêt DFS, qui n'est de fait connecté à aucun autre noeud. On a, à ce stade, deux noeuds (y et u) qui forment chacun une composante connexe où ils sont seuls.

— Finalement, on applique DFS(x). Encore une fois, le seul somme accessible u est déjà visité, donc le parcours s'arrête immédiatement. On ajoute alors x à la forêt DFS, seul. On a ainsi trois composantes connexes qui ne contiennent qu'un noeud, que sont y, u et x. Comme tous les sommets de V ont été visités, le parcours DFS global s'arrête et on a bien réussi à isoler u dans une composante connexe de la forêt DFS.

Question 3 On considère un digraphe **acyclique** $D = (V, E)$, et deux sommets s et $t \in V$. Écrire le pseudo-code d'un algorithme NOMBRE-CHEMINS(s, t), qui retourne le nombre de chemins allant de s à t. Cet algorithme devra avoir une **complexité asymptotique linéaire**.

Correction : Il s'agissait de l'exercice 22.4-2 de l'ouvrage de référence. Voici la correction qu'ils fournissent :

L'algorithme 1 fonctionne comme suit. L'attribut $u.paths$ du nœud u indique le nombre de chemins de u à v, en supposant que v est fixe tout au long du processus. Pour compter le nombre de chemins, nous pouvons additionner le nombre de chemins qui partent de chacun des voisins de u. Comme nous n'avons pas de cycles, nous ne risquons jamais d'ajouter un nombre de chemins seulement partiellement complétés. De plus, nous ne pouvons jamais considérer deux fois la même arête au cours des appels récursifs. Par conséquent, le nombre total d'exécutions de la boucle for sur tous les appels récursifs est $O(|V| + |E|)$. L'appel à SIMPLE-PATHS(s, t) donne le résultat souhaité.

Algorithm 1 Simple-Paths

Require: s and $t \in V$

```
1: if s == t then return 1
2: else if s.paths ≠ None then return s.paths
3: else
4:     for each w ∈ Adj[s] do
5:         s.paths = s.paths+SIMPLE-PATHS(w, s)
6:     end for
7: return s.paths
8: end if
```